LLM-Powered SOC Assistant

A Natural Language Interface for Open-Source Security Monitoring

Progress Report 4

Umesh Kalupathirannehelage – 300389749

Applied Research Project

CSIS4495-071

1. Work Log Table

Student Name: Umesh Kalupathirannehelage

Date	Hours	Description of Work Done
Oct 28 2025	2	Implemented hourly cron-based ETL automation using a custom /usr/local/bin/run_soc_etl.sh script to fetch and load logs automatically.
Oct 30 2025	2	Added a new /etl/status API endpoint to report the last and next scheduled ETL run times for both manual and cron executions.
Oct 31 2025	2	Enhanced the frontend (UI) to display *"Last Updated"* and *"Next Update"* timestamps synced with backend status.
Nov 1 2025	2.5	Debugged Wazuh log ingestion and PostgreSQL pipeline issues; ensured consistent updates from the hourly ETL cron job.
Nov 2 2025	1.5	Restructured backend to write last_run.txt after successful manual ETL runs, enabling timestamp refresh in UI.
Nov 4 2025	2	Refined main.py logic to unify cron and manual ETL reporting; optimized error handling and timeout recovery for /etl/run.
Nov 5 2025	2.5	Integrated auto-refresh functionality in UI for ETL status; ensured dynamic updates every 60 seconds; verified data freshness via API calls.
Nov 7 2025	2	Improved UI consistency and responsiveness; adjusted layout for chart and fetch buttons; added color differentiation for "Fetch Logs" and "Reset".
Nov 8 2025	1.5	Validated cron + manual ETL flow end-to-end, tested error cases, and finalized documentation updates for Progress Report 4.

2. Description of Work Done

Oct 23- Nov 8, 2025

During this reporting period, my primary focus was on automating the ETL ingestion process, improving system monitoring, and enhancing frontend usability to create a more autonomous SOC dashboard.

At the beginning of this phase, I developed and deployed a cron-based ETL automation script (run_soc_etl.sh) under /usr/local/bin/, configured to run every hour via crontab. This script triggers data ingestion from Wazuh's alerts.json into PostgreSQL, ensuring the SOC database remains continuously updated without manual intervention. Alongside this automation, I implemented a dedicated /etl/status API endpoint to provide real-time visibility into the system's operation displaying both the *last ETL execution time* and the *next scheduled run*.

To complement the backend automation, I enhanced the frontend UI to dynamically display these timestamps under the "Last Updated" and "Next Update" fields. The dashboard now automatically fetches status updates and synchronizes them after every successful ETL execution. In addition, I integrated a loading spinner during query execution and improved visual clarity by applying distinctive colors to critical action buttons such as "Fetch Logs" and "Reset."

Further refinements included restructuring the project directory for maintainability, placing assets like CSS and JavaScript into dedicated folders (ui/css/ and ui/script/). The backend (main.py) was enhanced to unify timestamp tracking between manual and cron-based ETL runs, while the UI (index.html) was updated to reflect real-time ETL updates and ensure a smooth user experience.

Toward the end of this period, I validated the AWS instance integration, ensuring successful communication between the Wazuh Manager, PostgreSQL database, and the FastAPI backend. These changes collectively improved automation, reduced manual maintenance, and elevated the overall reliability of the SOC Assistant application.

Under the implementations folder,

- **main.py (updated)** – Added cron integration and timestamp synchronization logic; introduced /etl/status endpoint; ensured both cron and manual ETL runs update last_run.txt for UI visibility.
- **ui/index.html (updated)** – Integrated "Last Updated" and "Next Update" display; added loading spinner; highlighted "Fetch Logs" and "Reset" buttons with new color themes.
- **/cron/** – Created folder containing the automation script run_soc_etl.sh and crontab configuration for hourly ETL ingestion.

Under the documents folder,

- Project_UKa749_Report4.pdf – This progress report document.